NOVA IMS INFORMATION MANAGEMENT SCHOOL

Variables and data types

Computation I · Lecture 02

Karina Brotto Rebuli
krebuli@novaims.unl.pt

Twenty-two, remember?

Here it is again

```
print("2" + "2")
```

```
>>> 22
```

LIVE

A program that adds 3 and 7

It runs. Nothing goes wrong.

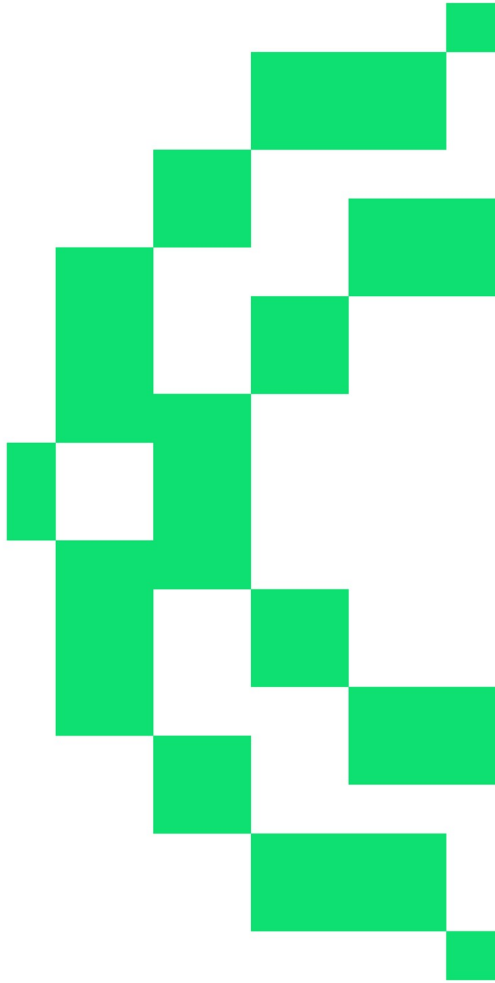
```
a = input("First number: ")
b = input("Second number: ")

print("Sum:", a + b)
```

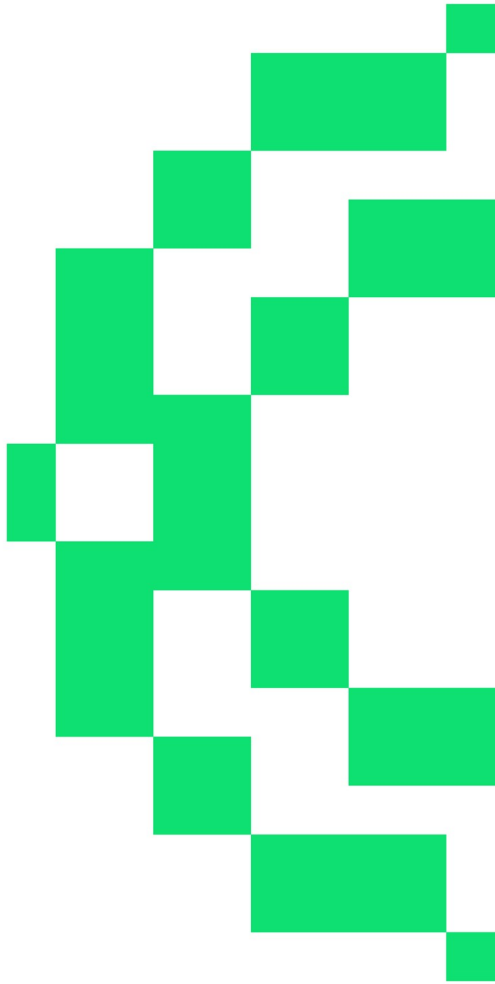
```
First number: 3
Second number: 7
Sum: 37
```

Three plus seven is not thirty-seven.

No crash. No warning. No line number. This is last week's third kind of error, and today you find out what is underneath it.



```
"""The type decides what your operators mean."""
```



Ada Lovelace

1843 — The first algorithm intended for a machine, a century before one was built. She had the idea this lecture is about.

The problem: everyone around her saw a machine for arithmetic. She asked what would happen if the symbols it moved around stood for something other than numbers — for letters, or notes of music.



Image source

Three questions today

What a name holds — and what it may be called.

What a value is in your computer — and how to convert its type.

What your operators mean.

01.

What a name holds

And why the equals sign is lying to you

Why bother with a name at all

The tax rate appears three times on the left, once on the right — as a variable

```
print(100 * 0.23)
print(250 * 0.23)
print(900 * 0.23)
```

```
tax_rate = 0.23

print(100 * tax_rate)
print(250 * tax_rate)
print(900 * tax_rate)
```

What a variable really is

Three parts you see, two Python hides

```
age = 19
```

One statement creates one object and binds a name to it.

01 · NAME	02 · TYPE	03 · VALUE
age	**int**	**19**
The unique identifier you write in your code.	What kind of data it is, and what you can do with it.	The data stored right now. It can change.

WHAT YOU SEE ---------- WHAT PYTHON HIDES

MEMORY ADDRESS

`0x7ffd4a09c3e8`

Where the object lives. Python assigns it for you.

RESERVED SPACE



A strip of bytes on the hardware, held for as long as the object is needed.

This is not mathematics

Read it as maths, and it is impossible

```
x = 5
x = x + 1

print(x)
```

```
>>> 6
```

The equals sign is an instruction, not a claim

Right side first, then its final result is assigned to the name on the left

1. Work out whatever is on the right of the `=`.

2. Give that result to the name on the left.

Which is why `x = x + 1` makes perfect sense, while `x + 1 = x` and `1 = x` are not even legal.

LIVE

A shortcut you will see everywhere

Same instruction, fewer characters

```
count = 0
count = count + 1
count += 1 # this is exactly count = count + 1

print(count)
```

Trace table

Run the program on paper — one row per instruction, one column per name

```
x = 4
y = x + 2
x = x * 2
y += 1
print(x, y)
```

x	y	screen
4		
4	6	
8	6	
8	7	
8	7	>>> 8 7

Naming rules

Two of these are law, the rest is courtesy

Legal — letters, _ and digits (but must *not start* with a digit, and *must not be a digit alone*).

Case matters — `total`, `Total` and `TOTAL` are three different names to Python.

Convention — lowercase with underscores: `tax_rate`, `first_number`.

Worth saying — `x` is legal and almost always a bad idea. You met the reason last week.

Thirty-five words you cannot use

They already mean something to Python

```
False      None       True       and        as         assert
async      await      break      class      continue   def
del        elif       else       except     finally    for
from       global     if         import     in         is
lambda     nonlocal   not        or         pass       raise
return     try        while      with       yield
```

LIVE

This is legal, and it is a disaster

In theory, you can use it — Python will not stop you from renaming its own tools

```
input = 5

name = input("Your name: ")
```

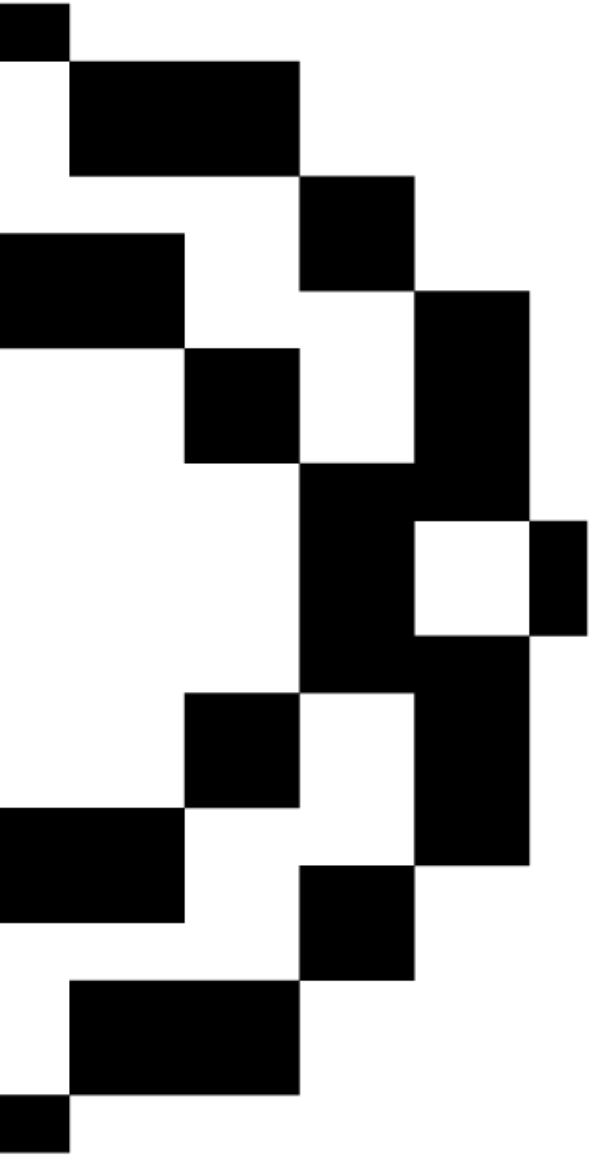
```
TypeError: 'int' object is not callable
```

What does this print?

```
x = 5
y = x
x = 10

print(y)
```

```
>>> 5
```



02.

What a value is

Four kinds, and they behave differently

Every value has a kind. Python calls it a type.

And the type comes with a set of legal operations

int — whole numbers. `42`, `0`, `-7`

float — numbers with a decimal point. `3.14` , `0.5` , `0.0`, `-2.0`

str — text, always in quotes. `"hello"`, `"42"`, `""`

bool — only two values. `True` and `False`

Four variables in memory

Different types take up different amounts of space

flag	bool
True	

age	int
19	

price	float
4.5	

name	str
"Ana"	

WHAT YOU SEE — WHAT PYTHON HIDES

RESERVED SPACE IN MEMORY



flag age price name free

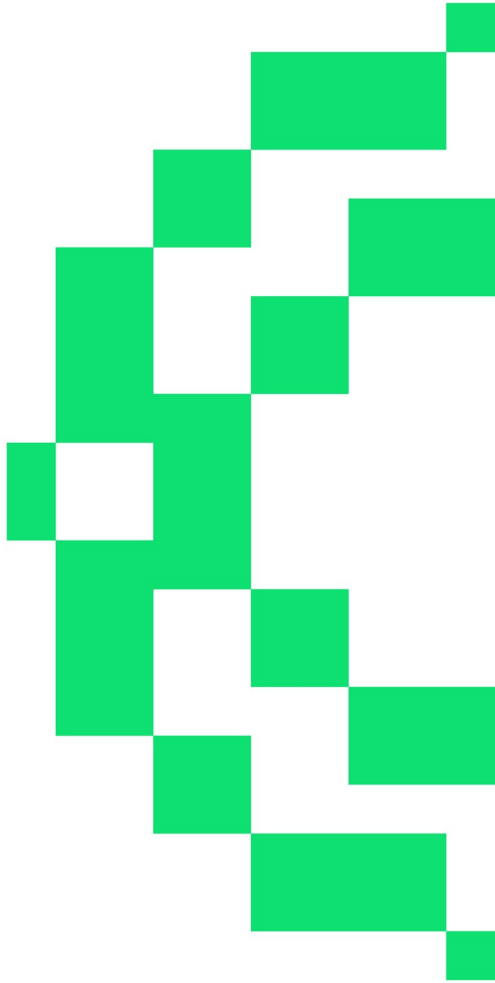
A bool needs the least space. A float always takes the same fixed block.

A string grows with its content: one cell per character. Sizes here are relative, not literal.

You never declared a single type today.

Python looked at the value, and knew.

Assignment creates the name, and the type arrives with the value — *implicitly*.



You can always ask

The type was never hidden — you just never asked

```
print(type(3))
print(type(3.0))
print(type("3"))
print(type(True))
```

```
>>> <class 'int'>
>>> <class 'float'>
>>> <class 'str'>
>>> <class 'bool'>
```

LIVE

Plus is not always an addition

The plus sign does two different jobs

```
print(2 + 2)
print("2" + "2")
```

```
>>> 4
>>> 22
```

The same plus sign, refused

Between text and a number, it has no agreed meaning at all

```
age = "19"

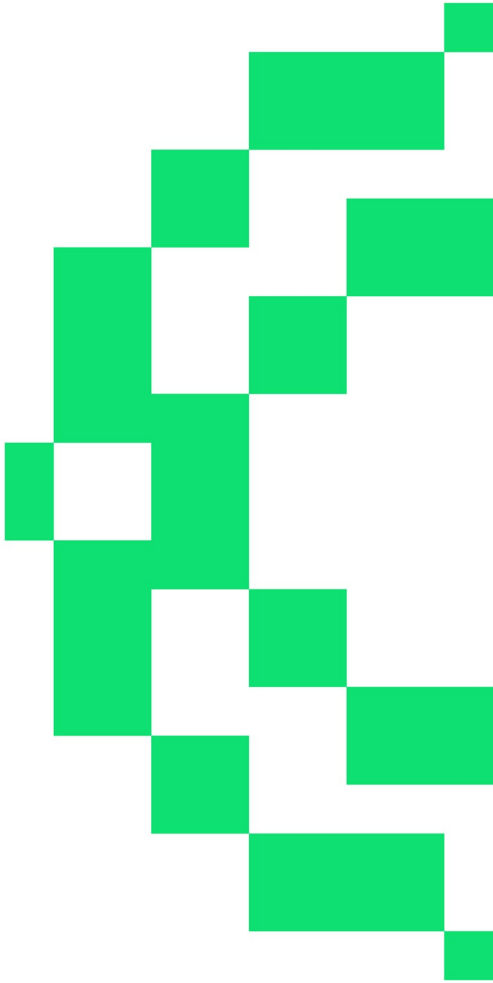
print(age + 1)
```

```
TypeError: can only concatenate str (not "int") to str
```

One stayed silent. One refused.

Both were the types talking.

The difference is only whether the operation happened to mean something.



Turning one type into another

Three functions, and you will use them constantly

int("3") → `3` — text that looks like a whole number becomes an integer

float("3.5") → `3.5` — same, with decimals (floating point)

str(3) → `"3"` — a number becomes text

Careful — `int("hello")` fails, loudly. Not everything can be converted.

LIVE

Now fix the 3 + 7 program

One function, in two places

```
a = input("First number: ")
b = input("Second number: ")

print("Sum:", int(a) + int(b))
```

```
First number: 3
Second number: 7
Sum: 10
```

Primitive and complex

Today's four hold one value each — almost

Primitive

Holds a single value.

`int`, `float`, `str`, `bool` — today's four.

Complex

Holds other values inside it. One name, many values.

Same anatomy — a name, a type, a value. Only the value grows.

Actually, `str` is the odd one: it can be taken apart. That story is told later in the course.

Your first complex type

A name, a type, a value — the anatomy is the same

```
grades = [15, 12, 18, 9]

print(type(grades))
```

```
>>> <class 'list'>
```

The type belongs to the value, not to the name

Python has dynamic typing, so the same name can hold anything, at any time

```
thing = 50
print(type(thing))

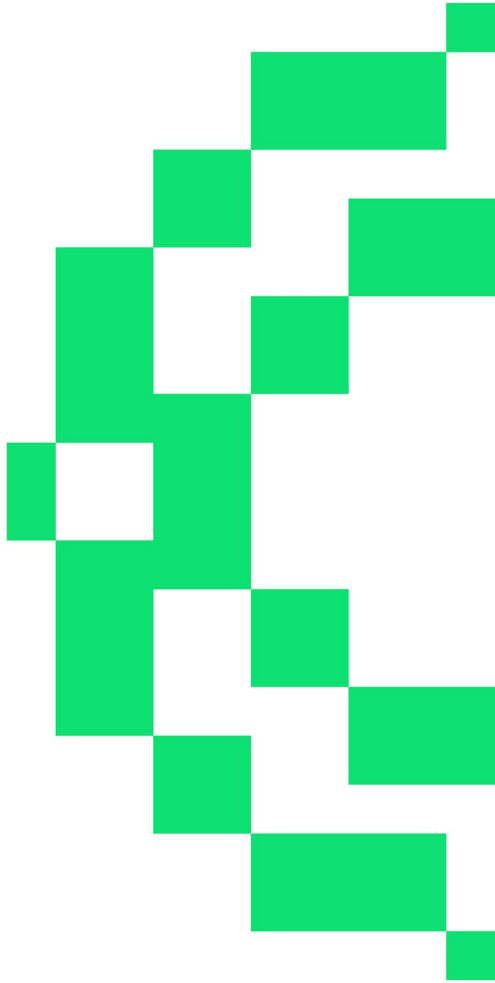
thing = "fifty"
print(type(thing))
```

```
>>> <class 'int'>
>>> <class 'str'>
```

Python never asked you to declare a type,

and it will never warn you when one changes.

That is why it is quick to write, and why the errors are quiet.



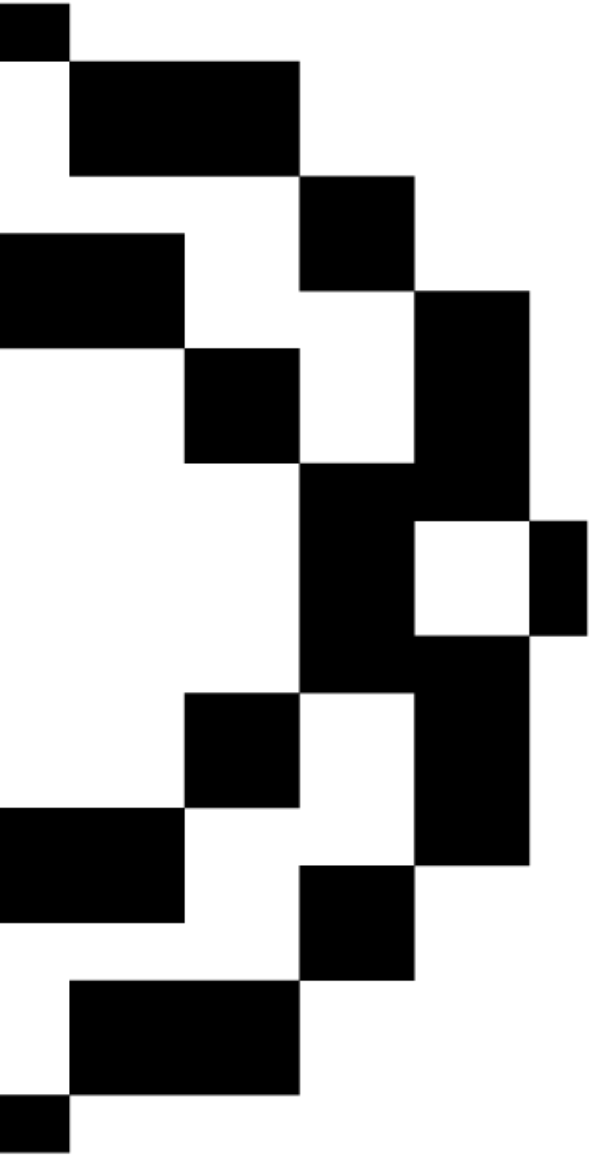
Where is this one broken?

It runs. Nobody complains.

```
price = "5"
quantity = 3

print("Total:", price * quantity)
```

```
>>> Total: 555
```



03.

What your operators mean

And the order Python calculates in

Arithmetic operators

Three of them are new

```
print(7 + 2, 7 - 2, 7 * 2)
print(7 / 2)
print(7 // 2, 7 % 2, 7 ** 2)
```

```
>>> 9 5 14
>>> 3.5
>>> 3 1 49
```

Division always answers in float

Even when it comes out integer value — this is consistency

```
print(10 / 5)
print(type(10 / 5))
```

```
>>> 2.0
>>> <class 'float'>
```

Floor division

Floor means truncate round down the division result — down toward negative infinity

```
print(7 / 2)
print(7 // 2)
print(7 // 4.5)
```

```
>>> 3.5
>>> 3
>>> 1.0
```

```
print(-7 / 2)
print(-7 // 2)
print(-7 // 4.5)
```

```
>>> -3.5
>>> -4
>>> -2.0
```

Modulo

The integer remainder of a floor division

```
print(7 % 2)
print(8 % 2)
print(23 % 10)
```

```
>>> 1
>>> 0
>>> 3
```

```
print(-7 % 2)
print(-7 % -2)
print(7 % -2)
```

```
>>> 1
>>> -1
>>> -1
```

One question all division operators refuse

Arithmetic division, floor division, and the remainder will always fail if the denominator is zero

```
print(10 / 2)
print(10 / 0)
print("done")
```

```
>>> 5.0
ZeroDivisionError: division by zero
```

Power operator

Multiplication, repeated — and it grows fast

```
print(2 ** 3)
print(2 ** 10)
print(2 ** 0.5)
```

```
>>> 8
>>> 1024
>>> 1.4142135623730951
```

Powers associate to the right

The one operator that reads backwards

```
print(2 ** 3 ** 2)
print(2 ** (3 ** 2))
print((2 ** 3) ** 2)
```

```
>>> 512
>>> 512
>>> 64
```

What does this print?

Commit before we run it

```
print(10 * 4 - 3)
print(10 * (4 - 3))
print(2 ** 2 ** 3)
print(2 ** (2 ** 3))
```

```
>>> 37
>>> 10
>>> 256
>>> 256
```

Who goes first

Except for the parentheses, it is Python's order, not yours

first

`( )`	parentheses
`**`	powers
`*  /  //  %`	multiplication and division
`+  -`	addition and subtraction

last

When int meets float

The int is promoted to float — nothing is asked

```
print(2 + 3.0)
print(type(2 + 3.0))
print(10 // 3, 10.0 // 3)
```

```
>>> 5.0
>>> <class 'float'>
>>> 3 3.0
```

What comes out, by what went in

One operator is the exception

`int` op `int`	→	`int`
`int` op `float`	→	`float`
`float` op `float`	→	`float`

A `float` anywhere makes the answer `float`.

---

anything `/` anything	→	`float`

And `/` makes it float even when nothing was.

LIVE

Python ships with a toolbox for maths

You open it with import

```
import math

print(math.pi)
```

```
>>> 3.141592653589793
```

LIVE

Asking the toolbox

The dot reaches inside the module

```
import math

print(math.sqrt(2))
print(math.sqrt(9))
```

```
>>> 1.4142135623730951
>>> 3.0
```

Three ways to lose the decimals

Down, up, or to the *nearest even*

```
import math

print(math.floor(3.7))
print(math.ceil(3.2))
print(round(3.5), round(2.5))
```

```
>>> 3
>>> 4
>>> 4 2
```

Six ways to ask

Each one asks a question about two values; Python answers True or False

`==`	is this equal to that?
`!=`	is this different from that?
`<`	is this less than that?
`>`	is this greater than that?
`<=`	is this less than or equal to that?
`>=`	is this greater than or equal to that?

The bool type

Every answer is a value — and a value has a type

```
print(7 > 3, type(7 > 3))
print(7 == 7.0, type(7 == 7.0))
print(7 != 7, type(7 != 7))
print(True, type(True))
```

```
>>> True <class 'bool'>
>>> True <class 'bool'>
>>> False <class 'bool'>
>>> True <class 'bool'>
```

You have seen this line before

This morning it was illegal — now you can read why

```
1 = x
```

```
SyntaxError: cannot assign to literal here.
Maybe you meant '==' instead of '='?
```

Diagnose this program

Calculate the total price with the tax added. Is the program below correct?

$$Total = price \times (1 + tax\ rate)$$

```
price = 45.00
tax_rate = 0.23

total = price * 1 + tax_rate
print("Total:", total)
```

```
>>> Total: 45.23
```

Three questions, answered

In one sentence each

A name

holds a value, and may be called almost anything — and the = sign is an assignment instruction, not a claim

A value

has a type, which is implicitly defined — but you can convert it

An operator

does what the types tell it to, in Python’s order — parentheses impose yours

Ada Lovelace

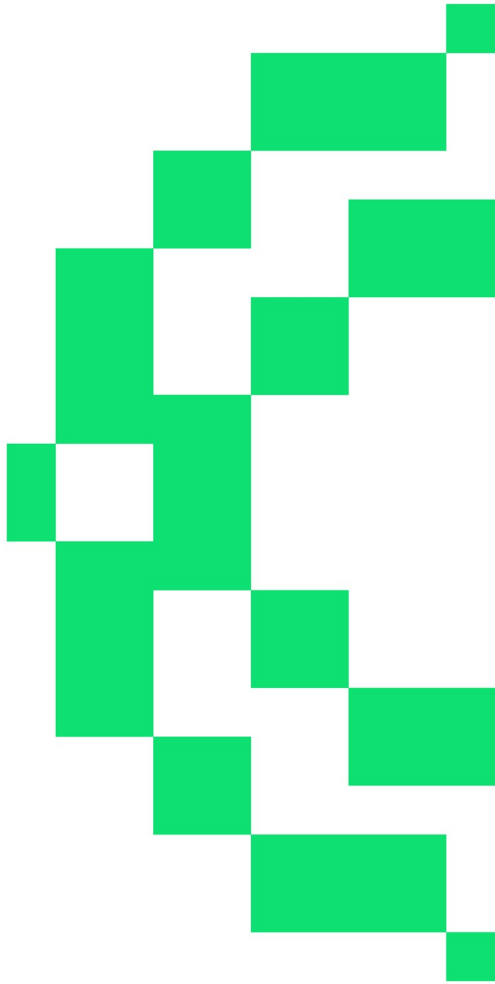
1843

The engine moved symbols. The symbols meant nothing until someone decided what they stood for.

“Two-two” and “three-seven” were never a mistake.

They were the answer to a different question.

You just had not told it which question you were asking.



Before the next theoretical lecture

On your own

Think Python — Sections 1.1 Arithmetic operators · 1.2 Expressions · 1.3 Arithmetic functions · 1.5 Values and types · 2.1 Variables · 2.3 Variable names · 2.4 The `import` statement · 2.5 Expressions and statements · 2.6 The `print` function · 2.9 Debugging · 5.1 Integer division and modulus.

One habit — when a program surprises you, print the `type()` before you print anything else.

Exercise 1 · Predict the type

Read the six expressions below — everything in them is this lecture's material. On paper, before any run: write the type each one has, in order. Then check each with print(type(…)) and compare.

```
7
7.0
"7"
7 > 3
2 + 3.0
10 / 5
```

Exercise 2 · Trace the assignments

Read the program below and give the final value of every variable, in the order a, b, c. On paper, before any run. Then add print(a, b, c) at the end, run it, and compare.

```
a = 4
b = a + 1
a = b * 2
c = a - b
b = c
```

Exercise 3 · Legal or illegal

Read the six candidate names below. For each one, say whether Python accepts it as a variable name, and why. Then try the ones you rejected in the Python console, and read what Python answers.

```
2nd_value
tax rate
_total
Total
class
print
```

Exercise 4 · Predict the value

Read the four expressions below — four of this lecture's operators. On paper, before any run: write the value each line gives, exactly as Python prints it. Then run them and compare.

```
7 / 2
7 // 2
7 % 2
2 ** 3
```

Exercise 5 · Place the parentheses

Rewrite each line below, adding one pair of parentheses so that it gives the value shown on its right. The order of operations is this lecture's material — the parentheses are how you overrule it. On paper first; then run each line and check.

```
3 + 5 * 2        →  16
10 - 4 - 3       →  9
2 ** 3 ** 2      →  64
```

Exercise 6 · How much do you pay?

This program should print the total to pay: the two prices added, then 4 off. Run it and read the message it stops with. Find the line to blame, say what each operator did before the stop, and make the program answer 10.

```
shampoo = "5"
soap = "9"
discount = 4

print("You pay:", shampoo + soap - discount)
```

Exercise 7 · How much do you pay with 10% off?

Write the line that prints what you pay: the two prices added, with 10% off the total. Start from the three values given in the panel — the prices arrive as strings, exactly as shown. Use this lecture's conversions and the order of operations.

```
shampoo = "5"
soap = "9"
discount = 0.10

# your line here
```

Exercise 8 · Find the side of the square

Write the one missing line: print the side of a square field whose area is 225 m². Start from the panel — the command to import the math module is already written for you. Print exactly the output below.

```
import math

area = 225
# your line here
```

```
>>> 15.0
```

Exercise 9 · How many boxes?

Write the one missing line: 121 books, 12 to a box — print how many boxes you need. Start from the data and the import given in the panel. Use this lecture's math module, and print exactly the output below — every book must travel.

```
import math

books = 121
per_box = 12
# your line here
```

```
>>> 11
```

NOVA IMS INFORMATION MANAGEMENT SCHOOL

```
input("Questions? ")
```

Acreditações e Certificações da NOVA IMS

eduniversal · QS STARS · ABET Computing Accreditation Commission · iSchools · EMOS · UNIGIS · USGIF · Erasmus Mundus Programme Education and Culture DG · Lapcer · IQNet

Cofinanciado por

PORTUGAL 2030 · 2030 Lisboa Programa Regional de Lisboa · Co-funded by the European Union · fct Fundação para a Ciência e a Tecnologia · PRR Plano de Recuperação e Resiliência · República Portuguesa · Financiado pela União Europeia NextGenerationEU

© Karina Rebuli, 2026

Exercise 10 · Predict, then run ★

Read the single line below. On paper, before any run:
1) Does it run or does it throw an error?
2) If it throws an error, which is the type of this error? Otherwise, what does it print?
3) If it prints something, which is the type of it?

```
print("10" < "9")
```